

DBMS

Transaction

- Finite sequence of database operations, constituting smallest logical unit of work from application perspective
- Properties:
 - Atomicity: Either all effects are reflected or none
 - Consistency: Guaranteed to yield correct database state
 - Isolation: Transaction is isolated from effect of concurrent transactions
 - Durability: Effects of commit are permanent even in case of failures
- Problems with Concurrent Transactions: Lost updates (Effect of transaction overwritten), Dirty reads (Reading invalid data), Unrepeatable read (Value retrieved twice with differing values)
- Serialisability: A concurrent execution of a set of transactions is serialisable if its execution is equivalent to some serial execution of the same set of transactions
- Two executions are equivalent if they have the same effect on the data
- DBMS supports concurrent executions for performance and enforces serialisability of concurrent executions for data integrity

Architecture

- 3-Tier Architecture / Levels of Data Abstraction
- External Schema: User or group-specific view on data
- Logical Schema: Logical organisation of data forming a data model, unified representation of all data, supports logical data independence and physical data independence
- Physical Schema: Organisation of data on disk and in memory, database as collections of fields, arrays, records, files, pages, etc.
- Logical Data Independence: Ability to change logical schema without affecting external schemas
- Physical Data Independence: Representation of data being independent from physical schema, can be changed without affecting logical schema
- Data Model: Set concepts for describing data
- Schema: Description of structure of DB using data model concepts
- Schema Instance: Content of a DB at a particular time

Relational Model

- Based on relations; tables with rows and columns
- Relation Schema: Specifies attributes (columns) and data constraints (domain constraints etc.)
- Domain: Set of atomic values
- Domain of attribute A_i is $dom(A_i)$, and each value v is either $v \in dom(A_i) \vee v = null$ (unknown or unspecified)
- Relation: Set of tuples, represented by relation schema $R(A_1, A_2, \dots, A_n)$
- Each instance of a schema R is $\subseteq \{(a_1, a_2, \dots, a_n) | a_i \in dom(A_i) \cup \{NULL\}\}$
- $R.A_i$ is attribute A_i of relation R
- Relational Database Schema: Set of relation schemas + data constraints
- Relational Database: Collection of tables
- Integrity Constraints: Conditions that restrict what constitutes valid data, checked by DBMS
- Structural Constraints: Domain constraints, Key constraints, Foreign Key constraints
- Structural: Inherent to data model, not dependent on application
- General Constraints: Dependent on specific application etc. triggers

Key Constraints

- Superkey: Subset of attributes that uniquely identifies a tuple in a relation
- Key: Superkey that is also minimal (No proper subset is a superkey)
- Every relation has at least 1 superkey
- Candidate Keys: Set of all keys for a relation
- Prime Attribute: Attribute of candidate key
- Primary Key: Selected candidate key for a relation, with added non-null constraint (entity integrity constraint), underlined in schema notation
- Foreign Key: Subset of attributes of relation A that refers to primary key of relation B
- Each foreign key in A must either be a null value or appear as a primary key in B
- Foreign keys can be self-referential

Considerations

- Structural integrity constraints do not cover application-independent constraints
- Integrity constraints are not mandatory
- Performance might decrease due to additional checking of constraints, could improve due to creation of indexes which speed up retrieval

SQL

- Structured Query Language
- Domain-specific and declarative language
- Interactive SQL: Directly writing SQL statements to an interface
- Non-interactive SQL: Application written in host language with interfaces
- Statement Level Interface: Mixture of host language and SQL statements
- Call Level Interface: Entirely written in host language and SQL statements are executed with function calls
- Interactive SQL: Directly writing SQL statements into interface such as Command Line or Graphical User Interfaces

Commands

- DDL: Data Definition (CREATE, ALTER, DROP, RENAME, TRUNCATE)
- DML: Data Manipulation (INSERT, UPDATE, DELETE, MERGE)
- DQL: Data Query (SELECT)
- DCL: Data Control (GRANT, REVOKE)
- TCL: Transaction Control (BEGIN, COMMIT, ROLLBACK, SAVEPOINT)

Create / Insert / Delete / Update

- Create:

```
CREATE TABLE Employees (  
    id INTEGER,  
    name VARCHAR(50),  
    age INTEGER,  
    role VARCHAR(50)  
)
```
- Basic Data Types: BOOLEAN, INTEGER, FLOAT8, NUMERIC [(p, s)], CHAR(n), VARCHAR(n), TEXT, DATE, TIMESTAMP..
- Insert Into:

```
INSERT INTO Employees VALUES (101, 'Sarah', 25, 'dev');
```
- Delete:

```
DELETE FROM Employees WHERE role = 'dev';
```
- Update:

```
UPDATE Employees SET age = age + 1 WHERE name = 'Sarah';
```

Three-Valued Logic

- Result of comparing with null is unknown, while result of arithmetic with null is null
- Values: True, False, Unknown

- Use IS NULL to check if null value
- Use IS DISTINCT FROM to check inequality, equivalent to $<>$ if non-null
- Conjunction: $\text{False} > \text{null} > \text{True}$
- Disjunction: $\text{True} > \text{null} > \text{False}$

Integrity Constraints

- Types of Constraints: Not-null, Unique, Primary key, Foreign key, General
- Column Constraint: Applies to a single column, specified at column definition
- Table Constraint: Applies to one or more columns, specified after column definitions
- Assertion: Stand-alone command
- Not-Null: Violated when there is a tuple where "id IS NOT NULL" evaluates to false

```
id VARCHAR(50) CONSTRAINT nn_id NOT NULL,
```

- Unique: Violated when for any two tuples, "(id1 $<>$ id2) or (pname1 $<>$ pname2)" returns false

```
id INTEGER CONSTRAINT u_id UNIQUE,  
CONSTRAINT u_alloc UNIQUE (eid, pname)
```

- Primary Key: Selected key uniquely identifying tuples, Prime attributes cannot be null

```
id INTEGER PRIMARY KEY,  
PRIMARY KEY (eid, pname)
```

- Foreign Key: Subset of attributes of A refer to primary key of B , violated when foreign key does not appear as primary key in referenced relation and is not null

```
FOREIGN KEY (eid) REFERENCES Employees (id),
```

- Behaviour can be extended with ON DELETE / UPDATE <action>
 - NO ACTION: rejects if it violates constraint
 - RESTRICT: similar, but can defer check of constraint
 - CASCADE: propagates to referencing tuples
 - SET DEFAULT: update foreign key to some default value
 - SET NULL: update foreign key to null
- CHECK constraint: Most basic general constraint, specifies Boolean expression that column values must satisfy

```
CONSTRAINT valid_lifetime CHECK (start_year <= end_year)
```

- Assertion: CREATE ASSERTION, used for arbitrary constraints, cannot modify data, and not linked to specific table, most RDBMS do not support
- All constraints can be specified named or unnamed
- All column constraints can be specified as table constraints other than not null
- Table constraints referring to a single column can be specified as a column constraint
- Column and table constraints can be combined

Deferrable Constraints

- By default, constraints are check immediately, and violations will cause rollbacks
- Can defer constraints to end of transaction
- Allows cyclic foreign key constraints
- No need to care about order of statements, and performance boost when constraint checks are bottleneck
- Troubleshooting is harder, data definition might be ambiguous, and queries might incur performance penalties when certain checks occur at run time

```
CONSTRAINT manager FOREIGN KEY (manager)
```

```
REFERENCES Employees (id)
DEFERRABLE INITIALLY IMMEDIATE/DEFERRED
...
SET CONSTRAINT manager DEFERRED;
```

Modifying Schema

- Alter Table: e.g. Add/Drop Columns/Constraints, change table specifications

```
ALTER TABLE Projects ALTER COLUMN start_year DROP DEFAULT;
ALTER TABLE Projects ADD COLUMN budge NUMERIC DEFAULT 0.0;
ALTER TABLE Teams ADD CONSTRAINT eid_fkey FOREIGN KEY (eid)
REFERENCES Employees (id);
```
- Drop Table: Without dependent objects, check if exists first; With dependent objects choose to cascade

```
DROP TABLE IF EXISTS Projects;
DROP TABLE Projects CASCADE;
```

Entity Relationship Model

- Data is described using entities and their relationships, with information described as attributes
- Data constraints can also be modeled
- Entity: Real world objects that are distinguishable from others
- Entity Set: Collection of entities of the same type, represented by rectangles, typically nouns
- Attribute: Specific information describing an entity, represented by a small circle
- Key Attributes: Uniquely identify entity, indicated by filled circle
- Derived Attribute: Derived from other attributes, indicated by dashed line
- Composite Key Attributes: Attributes which together uniquely identify each entity, indicated by additional connecting line
- Composite Attributes: Prefer splitting it up into components
- Multivalued Attributes: Refer to a set or list of values, can either be single valued attributes or dedicated entity set

Relationship Sets

- Relationship: Association between two or more entities
- Relationship Set: Collection of relationships of the same type, represented by diamonds, can have their own attributes, typically verbs
- Role: Descriptor of entity set's participation in relationship
- Degree: How many entity roles participate in a relationship (2 and 3 most common)

Relationship Constraints

- Cardinality: Upper bound of how many times entity can participate in relationship
- Many-Many, Many-One, One-One
- Participation: Lower bound to how many times entity can participate in relationship
- Indicated by (min, max) labels: User (0, n) makes (1, 1) Booking means each user can make multiple bookings, but each booking is only made by exactly one user

Dependency Constraints

- Weak entity set: Entity set without its own key, can only be uniquely identified by considering primary key of owner entity
- Identified by double-lined rectangles or diamonds
- Many to one relationship from weak entity set to owner entity set, must have (1, 1) attachment to identifying relationship
- Has partial key which uniquely identifies weak entity with context of owner entity
- Relation connects to attribute of owner entity

Aggregation

- Relationships between entity sets and relationship sets
- Treat relationships as higher-level entities, indicated by rectangle around them

Relational Mapping

- Mapping from ER model to relational schema
- Straightforward mapping from entity sets to tables
- Multivalued attributes: With fixed number of single-valued attributes, can decompose, if not create a new entity set with foreign key
- Relationship sets: Model in new table, with key attributes of all participating entity sets as well as own relationship attributes
- Use primary and foreign keys to capture cardinality and participation constraints
- Weak entity sets: Need foreign keys an ON DELETE/UPDATE
- Aggregation: Primary key of aggregation relationship, associated entity set, and descriptive attributes
- ER diagram should capture as many constraints as possible without imposing unnecessary ones, similarly for relational schema

More SQL

- Multiset / bag semantics (relational models are sets)
- Query is done with SELECT statement
- Basic Form:

```
SELECT [DISTINCT] target_list FROM relation_list
[WHERE conditions]
```

Simple Queries

- Use wildcard "*" to include all attributes
- Use "expr BETWEEN x AND Y" for basic value range conditions
- Use "AS" to rename attributes
- Use "DISTINCT" to eliminate duplicates; IS DISTINCT FROM must evaluate to true for at least one attribute
- WHERE clause imposes conditions
- "_" matches single character, "%" matches zero or more, can use regexs as well

Set Operations

- Union-Compatible: R and S have the same number of attributes, and corresponding attributes have same or compatible domains without implicit conversions
- UNION, INTERSECT, EXCEPT (Set difference) can be used to combine results; use ALL to not eliminate duplicates
- Column names follow left relation

Join Queries

- Cartesian Join: Return all possible pairs across relations
- Join: Cross product + attribute selection
- Inner Join: Cartesian Join + Selection Condition + Attribute Selection
- Natural Join: Join condition implied by attribute names, only defined for equality comparison, result only contains one instance of matching attributes, performed over al attributes, equivalent to Cartesian Join if there are no common attributes
- Outer Join: Inner Join but also returns dangling tuples which might not have matches
 - LEFT OUTER JOIN = INNER JOIN + all remaining from left table
 - RIGHT OUTER JOIN = INNER JOIN + all remaining from right table
 - FULL OUTER JOIN = INNER JOIN + all remaining from both tables

Subqueries

- In FROM clause: Consequence of closure property (result is also a table), must be enclosed in parentheses, table alias mandatory, column aliases optional

- Expressions: IN, EXISTS, ANY/SOME, ALL
- IN: Subquery must return exactly one column, can typically be replaced with inner join, NOT IN can typically be replaced with outer join, can manually specify subquery result
- ANY/SOME: expr op ANY (subquery), subquery must return exactly one column, expr is compared to each row with op
- ALL: Similar, but must be true for all subquery rows
- Correlated Subquery: Uses value from outer query, could lead to slow performance, if subquery has null value, ALL condition can never evaluate to true
- Naming ambiguities can cause problems, resolve using table aliases
- Scoping Rules: Table alias declared in subquery can only be used in it or its own subqueries
- If same table is declared in both a subquery and outside the innermost one is applied
- EXISTS: Checks is subquery returns at least one tuple
- Scalar Subquery: Subquery returns a single value, can be used as expression in queries
- Row Constructors: Allow subqueries to return more than one attribute or column, must match subquery number of attributes

Sorting and Rank Based Selection

- Use ORDER BY with ASC or DESC
- Can sort by multiple attributes and with different orders in order
- LIMIT returns a portion of the table
- OFFSET specifies position of first tuple to be considered

More More SQL

Common SQL Constructs

Aggregation

- Compute a single value from a set of tuples
- e.g. MIN, MAX, AVG, COUNT, SUM
- null might be treated differently
- Data type affects applicability and return data type

Grouping

- Use GROUP BY to partition relation into groups
- Aggregation functions are applied over groups instead
- Two tuples belong to the same group if for all involved pairs of attributes it IS NOT DISTINCT
- If A_i of R appears in SELECT, it must either appear in GROUP BY or appear as input of an aggregation function in SELECT
- Or satisfy global condition where primary key of R appears in GROUP BY
- HAVING: Conditions check for each group defined by a GROUP BY clause, typically involve aggregate functions
- Must appear in GROUP BY clause, or as an input of the aggregation function for itself, or primary key of R appears in GROUP BY clause
- Evaluation: FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY > LIMIT/OFFSET

Conditional Expression

- CASE: Similar to switch statement, list of WHEN THEN with ELSE base case, only one result to be returned
- COALESCE: Returns first non-NULL value in list of input arguments
- NULLIF: Returns NULL if both values are equal, if not returns first value

- Common Table Expression: Temporary named query, allowing structure for queries to improve readability
- WITH ctename AS ctebody ...
- View / Virtual Table: Permanently named query, result of a query is not permanently stored, can be used like normal tables
- Use CREATE VIEW AS ... to make a VIEW
- Can be used almost like a normal table, with restrictions for INSERT, UPDATE, DELETE
- Updatable View: Only one entry in FROM clause, no WITH, DISTINCT, GROUP BY, HAVING, LIMIT, OFFSET, UNION, INTERSECT, EXCEPT, ALL, aggregate functions

- SQL only supports existential quantification (EXISTS)
- Universal Quantification: Transform query to EXISTS using logical equivalences
- Recursive Queries: Each query for X stops requires X joins
- Use CTEs for recursive queries

Relational Algebra

- Formal method to process and query relations
- Operators:
 - Unary: Selection (σ), Projection (π), Renaming (ρ)
 - Binary: Union ($\cup$), Intersection ($\cap$), Difference ($-$), Product ($\times$), Inner Join ($\bowtie$), Outer Join ($\ltimes, \ltimes, \ltimes$)
- Operands = Relations, Operators = Transforming input relations into output relation
- Closure: All inputs and outputs of operators are relations, allows for nesting of relational operators forming expressions and query trees

Renaming

- Renaming Relation: $\rho(R, S)$
- Renaming Attributes: $\rho(R, R(a_1 \rightarrow b_1, \dots))$
- Renaming relation and attributes: $\rho(R, S(a_1 \rightarrow b_1, \dots))$
- New names are only valid within scope of expression

- $\sigma_c(R)$ selects all tuples from relation R satisfying c (evaluates to true)
- Precedence: Parentheses, Comparison, Negation, Conjunction, Disjunction

- ## Projection

- Operator: $\pi_{\ell}(R)$ projects all attributes of a relation specified in ℓ , with order mattering
- ℓ does not allow duplicates, and must be attributes of R

Binary Operators

Set Operators

- UNION, INTERSECT, EXCEPT
- Relations must be union-compatible: Have same number of attributes, and corresponding attributes have the compatible domains

Cross Product

- Relation formed by combining all pairs of tuples
- Most queries requiring cross product also require attribute selection

Joins

- Join operators combine cross product, selection, and possibly projection
- Avoid generating all tuples of cartesian product
- Inner Joins: Include only tuples satisfying condition
- Outer Join: Include tuples that do not satisfy the condition

Inner Joins

- θ -join: $R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$
- Equi Join: θ -join but only the $=$ operator can be used
- Natural Join: Equi Join is performed over all common attributes, output does not duplicate common attributes, more formally
 $R \bowtie S = \pi_{\ell}(R \bowtie_{c_{\rho}(b_1 \leftarrow a_{i_1}, \dots)}(S))$ where A is the set of common attributes, c selects common attributes, and ℓ is the list of all attributes in R and those in S but not in R

Outer Joins

- Dangle: $dangle(R \bowtie_{\theta} S)$ = set of dangling tuples in R wrt to $R \bowtie_{\theta} S$
- Null: $null(R)$ = n -component tuple of null values where n is the number of attributes of R
- Left: $R_1 \bowtie_{\theta} R_2 = R_1 \bowtie_{\theta} R_2 \cup (dangle(R_1 \bowtie_{\theta} R_2) \times \{null(R_2)\})$
- Right: $R_1 \bowtie_{\theta} R_2 = R_1 \bowtie_{\theta} R_2 \cup (\{null(R_1)\} \times dangle(R_2 \bowtie_{\theta} R_1))$
- Full: $R_1 \bowtie_{\theta} R_2 = R_1 \bowtie_{\theta} R_2 \cup (dangle(R_1 \bowtie_{\theta} R_2) \times \{null(R_2)\}) \cup (\{null(R_1)\} \times dangle(R_2 \bowtie_{\theta} R_1))$
- Natural Outer Join: Only equality is used, join performed over all attributes in common, output contains common attributes only once
- Use $\equiv$ to check equality with $null$

SQL Procedures and Functions

- Creation and execution of code directly within the database
- Procedures do not have returns while functions do

SQL Functions

```
CREATE OR REPLACE FUNCTION calc_age(dob DATE)
RETURNS INTEGER AS $$
DECLARE
    years INTEGER;
BEGIN
    years := <something>;
    IF <something>
    THEN years := years - 1;
    END IF;
```

- Can use SELECT on result of invocation like normal
- Can use WHILE condition LOOP ... END LOOP
- Can use RAISE to provide messages of different level, RAISE EXCEPTION aborts current transaction
- DEBUG, LOG, INFO, NOTICE, WARNING, EXCEPTION
- Can return a single tuple from a table by specifying table name as return type
- Can specify OUT parameter and return a RECORD, there are also INOUT parameters
- Input must match number of IN parameters, SELECT must match OUT columns
- Can return a table similarly but use SETOF table
- Can return a table using TABLE(...) with the specified attributes
- Can use FOUND to check result of previously executed query

SQL Procedures

- Function that does not return any value
- Use `SELECT INTO` var to set local variable

```
CREATE OR REPLACE PROCEDURE
download(cid VARCHAR(16), gname VARCHAR(32), gver CHAR(3))
AS $$
DECLARE age INTEGER;
BEGIN
    SELECT calc_age(c.dob) INTO age FROM customers c
    WHERE c.customerid = cid;
    IF gname <> 'Domainer' OR AGE >= 21 THEN
        INSERT INTO downloads VALUES (cid, gname, gver);
    END IF;
END;
$$ LANGUAGE plpgsql;
```

- Cursor: Pointer to row, can move in different directions and modes, must be closed
- NEXT / FORWARD, LAST, PRIOR / BACKWARD, FIRST, ABSOLUTE n, RELATIVE n, SCROLL, NO SCROLL
- Declare, Open, Fetch, Check if found, Close
- Fetch performs a move then retrieval
- Can use RETURN NEXT in loop to build output TABLE

```
CREATE OR REPLACE FUNCTION max_increase(gname VARCHAR(32))
RETURNS NUMERIC AS $$
DECLARE
    cur CURSOR (vname VARCHAR(32)) FOR
    SELECT g.price FROM games g WHERE g.name = vname
    ORDER BY g.version ASC;
    res NUMERIC; prev NUMERIC; curr NUMERIC;
BEGIN
    OPEN cur(vname := gname);
    res := 0; prev := 0;
    LOOP
        FETCH cur INTO curr;
        EXIT WHEN NOT FOUND;
        IF (curr - prev) >= res THEN res := (curr - prev); END IF;
        prev := curr;
    END LOOP;
    CLOSE cur;
    RETURN res;
END; $$ LANGUAGE plpgsql;
```

Triggers

- Stable Function: Always returns the same result given the same input
- CHECK requires functions that are immutable or stable, and cannot be deferred
- Trigger: Procedure or function executed when database event occurs on table
- Trigger checks if event occurred while trigger function is the action executed

```
CREATE CONSTRAINT TRIGGER tr21
AFTER INSERT OR UPDATE ON downloads
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE PROCEDURE fr21();
```

```
CREATE OR REPLACE FUNCTION fr21()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT c.customerid
        FROM customers c NATURAL JOIN downloads d
        WHERE d.name = 'Domainer'
        AND EXTRACT(year FROM AGE(c.dob)) < 21
    ) THEN
        RAISE EXCEPTION 'Underaged!'; -- STOP!
    END IF;
    RETURN NEW;
END;
```

- Trigger Event: Can be INSERT, UPDATE, DELETE
- Trigger Timing: Check BEFORE or AFTER
- Trigger Granularity: Changes to each ROW or STATEMENT
- Only AFTER trigger can be deferred, need keyword CONSTRAINT
- Return has no effect on AFTER trigger, but we can use it to modify rows being operated on for BEFORE
- OLD / NEW refer to state of row before / after operation
- TG_OP stores row operation
- INSTEAD OF ... ON view can be used to perform operation on a VIEW
- Can use WHEN to decide when trigger is activated, but cannot have SELECT; OLD for INSERT; NEW for DELETE; or INSTEAD OF
- Granularity: Can be FOR EACH ROW or STATEMENT
- ROW: AFTER / BEFORE (tables), INSTEAD OF (views)
- STATEMENT: AFTER / BEFORE (tables and views), INSTEAD OF (not allowed)
- Order of Activation: BEFORE STATEMENT, BEFORE ROW, AFTER ROW, AFTER STATEMENT, alphabetical order

Anomalies and Functional Dependencies

- Source of Anomalies: Some columns are uniquely determined by other columns
- Can be translated into an integrity constraint known as a functional dependency
- Functional Dependency: An instance r of relation R satisfies the functional dependencies σ of the form $X \rightarrow Y, X \subseteq R, Y \subseteq R$, iff two tuples of r agree on X implies they agree on Y
- $X \rightarrow Y$ means X functionally determines Y , or Y is functionally dependent on X
- Set Functional Dependency: A set of FDs is satisfied iff all FDs in Σ are satisfied
- Valid instance: Satisfies Σ
- Violation: Two tuples of r agree on X but not on Y
- Holds: A set of FDs Σ holds on a relation R if R is the set of valid instances which satisfy Σ
- Triviality: For $X \rightarrow Y, Y \subseteq X$, Non-Triviality: $Y \not\subseteq X$
- Completely Non-trivial: $Y \neq \emptyset, Y \cap X = \emptyset$, no common attributes
- Superkey: Let R be a relation, and $S \subseteq R$ be a set of attributes of R . S is a superkey of R iff $S \rightarrow R$, which is a superset of a key

- Candidate Key: $S \rightarrow R$, for all $T \subset S, T$ is not a superkey, it is a minimal superkey
- Primary Key: Candidate key chosen by designer
- Prime Attribute: Attribute that appears in some candidate key of R with Σ
- Closure of Σ : The closure of Σ, Σ^+ is the set of all functional dependencies logically entailed by the function dependencies in Σ
- Closure of S : The closure of $S \subseteq R, S^+$, is the set of all attributes functionally dependent on S
- Armstrong Axioms:
 - Reflexivity: $\forall X \subseteq R, \forall Y \subseteq R, ((Y \subseteq X) \Rightarrow (X \rightarrow Y))$ (trivial FDs)
 - Augmentation:
 $\forall X \subseteq R, \forall Y \subseteq R, \forall Z \subseteq R ((X \rightarrow Y) \Rightarrow (X \cup Z \rightarrow Y \cup Z))$ (add to both sides)
 - Transitivity:
 $\forall X \subseteq R, \forall Y \subseteq R, \forall Z \subseteq R, ((X \rightarrow Y \wedge Y \rightarrow Z) \Rightarrow (X \rightarrow Z))$ (short-circuit)
 - Completeness: All functional dependencies that hold can be obtained using Armstrong axioms
- Attribute Closure (Algorithm 1):
0 = Sigma;
G = S;
while (X implies Y in 0) and (X subset G) {
 0 = 0 - {X implies Y}
 G = G union Y
}
return G
- Equivalent Σ : Two sets of FDs are equivalent iff they have the same closure, then they are both covers of each other
- Cover: F covers G if every FD in G can be derived from F
- Two sets of FDs are equivalent iff all FDs in one are a logical consequence of FDs in the other and vice versa
- Minimal: RHS of every FD is minimal, LHS of every FD is minimal, Σ is minimal
- A minimal cover is both minimal and a cover
- Minimal: RHS of every FD has one attribute, cannot remove any attribute from the LHS, and cannot remove any FDs from Σ
- Minimal Cover: Both minimal and a cover (equivalent)
- Canonical Cover: Σ is a canonical cover of Σ' if there is Σ_1 which is a minimal cover of Σ' and we produce Σ by regrouping all FDs with same LHS in Σ_1 ; we find a minimal cover and merge the RHS if LHS is equal
- Every set of FDs has a minimal cover which might not be unique
- Minimal Cover (Algorithm 2):
 1. Simplify RHS of every FD
 2. Simplify LHS of every FD
 3. Simplify set Σ
 - Step 2: For $X \rightarrow \{A\}, B \in X$ can be removed if X without B still implies A
 - Step 3: For $X \rightarrow Y$, it can be removed if it is still implied by the remaining FDs
- Canonical Cover (Algorithm 3): After Step 3, regroup all FDs with same LHS
- Tip 1: Start from smallest cardinality
- Tip 2: If an attribute does not appear in any RHS, it must be a part of all candidate keys

Boyce-Codd Normal Form

- Normal Forms: Recognise designs that enforce FDs by means of SQL constraints to protect against data anomalies
- Normalisation: Transform a poor design into a design that enforces FDs by means of main SQL constraints
- BCNF: A non-key field must provide a fact about the key, the whole key, and nothing but the keys

- A relation R with Σ is in BCNF iff for every FD $X \rightarrow \{A\} \in \Sigma^+, X \rightarrow \{A\}$ is trivial or X is a superkey
- Binary Decomposition: Pair of tables with $R = R_1 \cup R_2$, can be expanded to general decomposition
- Lossless-Join Decomposition: Natural join of two fragments equals the initial table
- A binary decomposition of R is lossless-join if $R = R_1 \cup R_2$ and $(R_1 \cap R_2) \rightarrow R_1/R_2$; find $R_{\cap}, R_{\cap}^+$, check if $R_1 \subseteq R_{\cap}^+$ or the other
- A general decomposition is lossless-join if there exists at least one sequence of binary lossless-join decomposition that generates that decomposition
- Projected Functional Dependencies: Σ' of projected FDs on R' from $R, R' \subseteq R$, is the set of FDs where $\forall X \rightarrow Y \in \Sigma^+, X \subseteq R' \wedge Y \subseteq R'$, both sides are contained, denoted $\Sigma|_{R'}$
 1. List all subsets of attributes
 2. Compute the attribute closures
 3. Intersect RHS with R'
 4. RHS - LHS
 5. Construct FDs
- Dependency Preserving Decomposition: $\Sigma^+ = (\Sigma|_{R_1} \cup \dots \cup \Sigma|_{R_n})^+$
- Algorithm: Repeatedly pick a violating FD and use it to decompose the relation into two
- Algorithm 4: Decompose violating $X \rightarrow Y$ (non-trivial and not superkey) into $R_1 = X^+, R_2 = (R - X^+) \cup X$ then repeat
- The algorithm is a lossless-join decomposition, and will terminate, might not be dependency-preserving

Third Normal Form

- BCNF might not be dependency preserving
- Third Normal form relaxes BCNF requirements for prime attributes
- 3NF: For every FD $X \rightarrow \{A\} \in \Sigma^+, X \rightarrow \{A\}$ is trivial, X is a superkey, or A is a prime attribute
- Algorithm 5: We can synthesise a schema in 3NF from a minimal cover of the set of FDs, for each FD $X \rightarrow Y$ in the minimal / canonical cover, create $R_i = X \cup Y$ unless it already exists or is subsumed
- If a key does not appear, pick any candidate key and create a relation with it
- The algorithm guarantees a lossless-join, dependency-preserving decomposition in 3NF
- $4NF \subseteq BCNF \subseteq 3NF \subseteq 2NF \subseteq 1NF, 1NF \not\subseteq 2NF \not\subseteq 3NF \not\subseteq BCNF \not\subseteq 4NF$

Revision

- Function: Use SELECT, Procedure: Use CALL
- DECLARE varname, datatype;
- Assign with varname := expression;
- Use IF THEN ELSIF THEN ELSE END IF
- LOOP EXIT label WHEN condition; statement; END LOOP
- WHILE condition LOOP statement END LOOP;
- CREATE TRIGGER name timing event ON table FOR EACH granularity WHEN condition EXECUTE FUNCTION func()
- BEFORE, AFTER, INSTEAD OF
- INSERT, DELETE, UDPATE
- FOR EACH ROW, FOR EACH STATEMENT
- OLD, NEW
- CONSTRAINT, DEFERRABLE, INITIALLY DEFERRED, INITIAIY IMMEDIATE, put before FOR EACH